

Cuadro comparativo - AcopioLeche

Cuadro comparativo: KMP vs Flutter vs React Native

Equipo AcopioLeche — Semana 1

Dimensión	Kotlin Multiplatform (KMP)	Flutter	React Native
Lenguaje y curva de aprendizaje	Kotlin. Es una curva de aprendizaje tranquila para aquellos que ya están familiarizados con Java/Kotlin (es útil si el grupo tiene experiencia previa en Android nativo o en cursos de Java); es necesario comprender conceptos multiplataforma (expect/actual).	Dart. Lenguaje nuevo para la mayor parte de los alumnos; curva media, pero bien documentado y con herramientas amigables.	TypeScript/JavaScript. Para aquellos que ya tienen conocimientos de React/web, la curva de aprendizaje es baja; el equipo debe saber manejar el ecosistema npm.
Estrategia de UI	Es posible, por un lado, conservar la UI nativa por plataforma o, por otro lado, compartirla con Compose Multiplatform (a partir de 2025). Esto brinda flexibilidad pero requiere más decisiones arquitectónicas.	Un único marco de UI (widgets propios), compartido completamente en todas las plataformas, que cuenta con un motor de renderizado propio (Impeller).	UI declarativa que se fundamenta en componentes React, convertidos a vistas nativas a través de la Nueva Arquitectura (Fabric).
Qué se comparte entre plataformas	Flexible: el equipo puede decidir compartir desde solo la lógica de negocio hasta casi todo el código (con Compose Multiplatform).	Casi toda la aplicación (interfaz de usuario + lógica) se distribuye automáticamente entre Android, iOS, la web y escritorio.	En su mayoría, la lógica y la interfaz de usuario se comparten, pero para las funciones particulares del dispositivo, generalmente dependen de módulos nativos de terceros.
Rendimiento	Cercano a nativo: En Android, compila directamente a código nativo; en iOS, compila a binarios nativos y tiene acceso directo a las APIs de la plataforma.	Motor de renderizado propio (Impeller/Skia); según informes de 2025-2026, se sitúa muy bien en interfaces complejas y animaciones suaves.	Históricamente, "el puente" había generado sobrecarga. JS-nativo; Hermes y la Nueva Arquitectura (Fabric + TurboModules) optimizaron significativamente el rendimiento durante el tiempo de ejecución y el arranque.
Ecosistema	Ecosistema en maduración: tiene librerías para red, base de datos, inyección de dependencias y UI, pero son menos numerosas que las de Flutter o React Native.	El más extenso en paquetes y widgets que ya están listos para usar, con el apoyo de una activa y extensa comunidad.	Expo 2026 ha estabilizado buena parte de esas dependencias, aunque depende mucho de módulos de terceros y el ecosistema de npm es muy grande.

Empresa respaldante	JetBrains (creadores de Kotlin y de los IDEs de IntelliJ), con colaboración de Google para Android.	Google.	Meta (antes Facebook), con aportes activos de Microsoft (React Native for Windows/macOS) y la comunidad de Expo.
Requisitos de entorno en Windows	Requiere Android Studio + JDK + Android SDK; para compilar a iOS se necesita una Mac (no es posible compilar el target iOS completo solo desde Windows).	Requiere Windows 7 SP1 de 64 bits o superior, ~400 MB de espacio para el SDK, y Android Studio (o solo el Android SDK) para el emulador.	Requiere Node.js, un gestor de paquetes (npm/yarn) y Android Studio con el SDK de Android para compilar y probar en emulador.

Fuentes (APA 7)

Java Code Geeks. (2026, 10 de febrero). *Kotlin Multiplatform vs. Flutter vs. React Native: The 2026 cross-platform reality*. <https://www.javacodegeeks.com/2026/02/kotlin-multiplatform-vs-flutter-vs-react-native-the-2026-cross-platform-reality.html>

JetBrains. (2026, 21 de julio). *Kotlin Multiplatform vs. React Native: A cross-platform comparison*. Kotlin Multiplatform Documentation. <https://kotlinlang.org/docs/multiplatform/kotlin-multiplatform-react-native.html>

Raza, S. A. (2025, 26 de diciembre). *Flutter vs React Native vs Kotlin Multiplatform (2026) – When to choose each*. <https://syedali.dev/flutter-vs-react-native-vs-kotlin-multiplatform-2026>

Dias, R. (2019, 26 de marzo). *Instalando o Flutter no Windows*. Medium - sysvale. <https://medium.com/sysvale/instalando-o-flutter-no-windows-7d19cfdae1b8>

Conclusión del equipo

Para AcopioLeche, nuestro equipo eligió Kotlin Multiplatform junto con Compose Multiplatform. Además de ser la tecnología establecida para el curso, nos brinda un desempeño casi nativo, el cual es crucial para trabajar offline-first en áreas rurales donde la conectividad es limitada. También tiene una curva de aprendizaje más asequible si se parte de una base previa en Java/Kotlin. En esta fase, no precisamos compilar a iOS, por lo que este requerimiento extra de KMP no constituye una limitación auténtica para el proyecto.

